Engineer to Excel

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

**SIMATS ENGINEERING**
**DESCRIPTIVE WRITTEN EXAM – ANSWERS**

**COURSE NAME:** Software Testing **STUDENT NAME:** B.Satya Hyndavi
**COURSE CODE:** CSA3712 **REG.NO:**192311178

**SECTION A – QUESTIONS**

1. **A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.**
2. **A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.**
3. **During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.**
4. **A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.**
5. **A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.**
6. **During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can**

**contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.**

7. **A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.**
8. **A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.**
9. **A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.**
10. **An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.**
11. **A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.**
12. **A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.**
13. **A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.**
14. **During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.**

15. **A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.**

16. **A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.**

17. **A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.**

18. **An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.**

19. **A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.**

20. **A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.**

21. **A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.**

22. **A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.**

23. **A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.**

24. **A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.**

25. **A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.**

26. **A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.**

27. **A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.**

28. **A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.**

29. **A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.**

30. **A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.**

## SECTION B – ANSWERS

### Answer 1

---

**Concept**

**Testing as an engineering activity** means testing is planned, designed, measured and repeated in a disciplined way, just like any other engineering work. It is not just "trying the software and seeing what happens".

**Problem in the scenario**

- Tests are random, so nobody knows what was covered.
- Results change from tester to tester.
- Important cases are missed, so defects go to the customer.

**Redesigned approach (steps)**

1. **Plan:** decide scope, objectives, risks, resources and schedule.
2. **Analyze requirements:** find what must be tested (test conditions).
3. **Design test cases:** use techniques like Equivalence Partitioning and Boundary Value Analysis.
4. **Prepare environment and test data.**
5. **Execute tests** and record actual results.
6. **Report defects** with clear steps, and track them to closure.
7. **Measure and improve:** check coverage and defect trends, then improve the next cycle.

**Random vs structured testing**

| Point | Random testing | Structured test design |
|---|---|---|
| Coverage | Unknown | Measured against requirements |
| Repeatability | Hard to repeat | Same tests can be re-run |
| Defect detection | Depends on luck | Targets likely defects |
| Result | Inconsistent | Consistent and reliable |

**Application**

For the startup's login module, random testing may try only one or two passwords. In structured design the tester writes cases for: valid login, wrong password, blank fields, minimum and

maximum password length, and account lock after 3 failed attempts. This finds more defects and gives the same result every time.

**Answer 2**

---

**Concept**

A **defined testing process** is a fixed set of activities followed in every project: planning, test design, execution, defect reporting and closure. In real-time systems like banking, a failure directly affects money and customers, so a fixed process is very important.

**Process followed**

1. Test planning and control
2. Test analysis and design
3. Test implementation and execution
4. Defect reporting and re-testing
5. Evaluating exit criteria and test closure

**Benefits with banking examples**

| **Benefit** | **Example in banking** |
|---|---|
| Fewer failures in production | Fund transfer that debits one account but does not credit the other is caught in testing |
| Early defect detection | Wrong interest calculation is found during review, not after month-end |
| Consistency | Every module (loans, ATM, net banking) is tested in the same way |
| Traceability | Every requirement such as "daily limit Rs. 50,000" has a test case |
| Lower cost and effort | Fixing before release is cheaper than fixing live data |
| Customer trust | Fewer wrong transactions and complaints |

**Conclusion**

A well-defined process makes testing planned and measurable, so software quality improves and failures in banking systems reduce.

**Answer 3**

---

**Concept**

In a structured process, testing is not done only at the end. Testing activities are linked to every SDLC phase (the idea of the V-model). Each development phase has a matching test activity.

**Testing in each SDLC phase**

| SDLC phase | Testing activity | E-commerce example |
|---|---|---|
| Requirements | Review requirements, plan acceptance tests | Check "free delivery above Rs. 500" is clear |
| Design | Review design, plan integration and system tests | Check cart-to-payment interface |
| Coding | Unit testing, code review | Test discount calculation function |
| Integration | Integration testing | Cart, payment and order modules together |
| System | Functional and non-functional testing | Full purchase flow, load on sale day |
| Deployment | Acceptance testing (UAT) | Business users confirm the flow |
| Maintenance | Regression testing | Check old features after a new offer is added |

**How process discipline improves quality**

- **Entry and exit criteria** make sure a phase is complete before the next one starts.
- **Standard documents** (test plan, test cases, defect reports) are used by everyone.
- **Traceability** links requirements to tests, so nothing is missed.
- **Defect tracking** shows which phase leaks defects.
- Integration and system defects are found in their own phase, not left for the customer.

## Answer 4

### Concept

The three terms are related but different.

| Term | Meaning | Login example |
|---|---|---|
| Error (mistake) | A human mistake made by the developer or analyst | Developer misreads "password minimum 8 characters" and writes length > 8 |
| Defect (bug/fault) | The wrong code or document caused by the error | The line "if length > 8" is present in the code |
| Failure | Wrong behaviour seen when the software runs | A valid 8-character password is rejected |

### Relationship

**Error → Defect → Failure.** A person makes an error, which puts a defect in the software, and when that code runs under certain conditions it causes a failure. Not every defect causes a failure (for example, defect in code that never runs).

### Classification of the given issue

- The tester sees expected output ≠ actual output during validation, so what is **observed** is a **failure**.
- The failure is caused by a **defect** in the login code or logic.
- The root cause of the defect is a developer **error**.

### Next step

Log it as a defect, let the developer debug and fix it, then re-test.

## Answer 5

### Concept

**Principle applied: Early testing.** Testing activities should start as early as possible in the lifecycle, because defects found early are cheaper and easier to fix.

### How the principle is applied

- Testers join requirement and design reviews.

- Static testing (reviews, walkthroughs, inspections) is done before coding.
- Test cases are designed while requirements are being written.
- Unit testing is done by developers right after coding.

**Cost of fixing a defect (relative idea)**

| Phase where found | Relative cost |
|---|---|
| Requirements | 1x |
| Design / Coding | 5x to 10x |
| System testing | 20x to 50x |
| Production | 100x or more |

**Example**

A requirement says "discount applies on orders above Rs. 1000". It is not clear whether Rs. 1000 itself is included. If the tester asks this in the review, it is fixed with one line of text. If it is found in production, wrong bills have to be corrected, customers refunded and code changed.

**Result**

Early testing improves effectiveness because defects are removed before they spread to design, code and test cases.

**Answer 6**

---

**Concept**

In the requirement phase, a tester acts as a **reviewer and a user of the requirements**. Testers look at requirements from the question "how will I test this?"

**How a tester contributes**

1. Review the SRS for **ambiguity, missing details, conflicts and untestable statements**.
2. Ask clarification questions to the business analyst.
3. Check that each requirement is **measurable and testable**.
4. Identify test conditions and write acceptance criteria.
5. Identify high-risk areas (for example patient safety).
6. Build a **Requirement Traceability Matrix (RTM)** to link requirements to tests later.

**Healthcare example**

Requirement: "System should alert the doctor if the dosage is high." The tester asks: what is "high"? For which age group? What type of alert? The requirement is corrected to give exact limits, such as "alert if dose exceeds 500 mg for a child under 12".

**Importance of early involvement**

- Ambiguities are removed before design and coding.
- Fixing a requirement is cheap compared with fixing code.
- Reduces rework and delays.
- Improves patient safety and software quality.
- Test planning starts early, so testing finishes on time.

## Answer 7

**Concept**

A defect can enter at any phase of the lifecycle, but **requirement defects spread the most** because design, code and test cases are all based on the requirements.

**How defects originate**

| Phase | How defects enter | Payroll example |
|---|---|---|
| Requirements | Unclear, missing or conflicting statements | "Overtime is paid extra" - rate not given |
| Design | Wrong interpretation of requirement | Designer assumes 1.5x for all days |
| Coding | Wrong logic, wrong assumption | Developer hard-codes 1.5x |
| Testing | Test cases built on the same wrong understanding | Tester also expects 1.5x |
| Maintenance | Change made without checking impact | Tax slab update breaks salary total |

**Analysis**

In the payroll case, the business meant 2x on holidays but the requirement was unclear. Every later phase carried the same mistake, so the output was wrong. The origin of the defect is the **requirement phase**.

**How requirement clarity prevents defects**

- Write requirements that are clear, specific, measurable and testable.
- Add examples with real numbers (basic Rs. 20,000, 10 overtime hours = Rs. ...).
- Hold reviews and walkthroughs with testers, developers and users.
- Use checklists and prototypes.
- Use change control and the RTM so changes are tracked.
- Get sign-off from the client.

**Answer 8**

---

**Classification**

The defect is a **functional defect (input-handling / boundary defect)** with **high severity**, because the system crashes.

**Justification**

- The software does not behave as the specification says for valid or limit input values.
- The crash happens only at the **edges** of the allowed range, usually because of wrong comparison in code (< instead of <=) or off-by-one errors.
- Crash means the whole function is unavailable, so severity is high or critical.
- It is also a **test design gap**, because boundary cases were not covered.

**Boundary Value Analysis (BVA)**

BVA is a black-box technique where test cases are chosen at the **edges of the input range**, because defects are most likely there. For range min to max, test: min-1, min, min+1, max-1, max, max+1.

**Example**

Field accepts age 18 to 60.

| Value | Type | Expected |
|---|---|---|
| 17 | Just below min | Rejected |
| 18 | Min | Accepted |
| 19 | Just above min | Accepted |
| 59 | Just below max | Accepted |
| 60 | Max | Accepted |
| 61 | Just above max | Rejected |

**How it would have prevented the issue**

Testing 17, 18, 60 and 61 would have triggered the crash in testing itself. The developer could then fix the condition before release.

**Answer 9**

---

**Concept**

A **defect repository** is a central database where all defects are stored with details such as ID, module, description, type, severity, priority, status, phase found, root cause and fix. It is the history of the product's quality.

**How it supports better test case design**

1. **Find weak modules:** modules with many defects get more test cases.
2. **Find common defect types:** if date and field-length defects repeat, add such cases to every test design.
3. **Add test cases for every fixed defect** and keep them in the regression suite.
4. **Use past defects as checklists** for new projects and releases.
5. **Find root causes** and design tests to catch that cause.
6. **Remove useless tests** that never find anything and add new ones (avoids pesticide paradox).

**Example**

If the repository shows that three releases had defects in the discount module for a wrong rounding of decimals, the team adds decimal and rounding cases to the discount test design. The same defect then cannot slip through again.

**Role in improving quality**

- Reduces repeated defects across releases.
- Helps in trend analysis and defect metrics.
- Improves planning and estimation.
- Gives management data for decisions.

**Answer 10**

---

**Concept**

A **functional defect** is a defect where the software does not perform a function as given in the functional requirements.

**Description for the ATM case**

- **Expected:** cash is dispensed and balance is reduced exactly once.
- **Actual:** transaction logic fails. For example, cash is not dispensed but money is deducted, or balance is not updated after dispensing.
- This is a functional defect because the withdrawal function does not meet its requirement.
- Severity is high since it involves money.

**Why it was missed**

- Limited test scenarios, only the normal withdrawal was tested.
- No tests for failures in the middle of a transaction.

**How it could have been prevented**

1. Review the requirements and design of the transaction logic.
2. Design tests using **decision tables and state transition testing** (card inserted, PIN verified, amount entered, cash dispensed, balance updated).
3. Include cases: insufficient balance, amount equal to balance, daily limit, amount not in multiples of 100, network or power failure during the transaction.

4. Do code review and unit testing of the transaction module.
5. Do integration testing with the bank server and the cash dispenser.
6. Run regression tests after every change.

**Answer 11**

---

**Concept**

**Defect clustering** is a testing principle which says that a **small number of modules contain most of the defects** (similar to the Pareto 80/20 rule).

**Using it to set testing priorities**

1. Collect defect data from earlier cycles and the repository.
2. Count defects per module and rank them.
3. Mark the top few modules as **high-risk / defect-prone**.
4. Give these modules **more test cases, deeper testing and more review time**.
5. Assign experienced testers to them.
6. Keep some basic testing on other modules, and re-check the ranking as new data comes.

**Example**

| **Module** | **Defects found** | **Priority** |
|---|---|---|
| Payment | 45 | High |
| Checkout | 25 | High |
| Search | 10 | Medium |
| Profile | 5 | Low |
| Others | 15 | Low |

Payment and checkout have 70 of 100 defects, so most of the effort goes there.

**How it improves efficiency**

- Limited time and people are used where defects are most likely.
- More defects found per hour of testing.
- Faster feedback to developers.
- Note: do not ignore other modules completely, because defects can move.

## Answer 12

### Concept

**Defect classification** means grouping defects by type, severity, priority, origin or other attributes, so they can be analysed and handled properly.

### Classifying the given defect

| Attribute | Classification | Reason |
|---|---|---|
| Type | Non-functional: **Performance** defect | Function works but response is slow under load |
| Severity | High | Many users affected in real use |
| Priority | High | Needs quick fix before peak usage |
| Likely origin | Design or coding | Slow queries, no caching, poor resource handling |
| Found by | Load / stress testing | |

### Other common defect classes (for reference)

- Functional, usability, security, compatibility, interface, data defects.

### How classification helps resolution

- Defect goes to the right person (for example database or backend team).
- Priority is decided correctly.
- The right test type is used (load and stress testing).
- Trend analysis shows which defect type is common.
- Prevention actions can be planned for that type.

## Answer 13

### Concept

**Regression testing** is re-testing the existing features after a change to confirm that nothing else has broken. Historical defect data tells us where to look.

### How the defect data is reused

1. Collect past defects with module, severity and cause.

2. Find **defect-prone modules** and frequently changed areas.
3. Select the test cases that **found defects earlier** and make them part of the regression suite.
4. Add a test case for **every fixed defect**.
5. Prioritize by severity and impact on business.
6. Remove old, obsolete tests and automate stable ones.

**Example**

If tax calculation failed in three releases, its tests are always in the regression pack and run first after every release.

**Improvement in coverage and efficiency**

| **Area** | **Benefit** |
|---|---|
| Test coverage | Risky and weak areas are always covered |
| Efficiency | Only useful tests are run, so less time is spent |
| Quality | Old defects do not come back |
| Cost | Fewer re-runs and fewer production issues |

**Answer 14**

---

**Role of a tester in reducing integration defects**

- Join design and **interface reviews** to understand how modules exchange data.
- Attend regular meetings with developers and share test status.
- Prepare integration test cases and test data early.
- Write **clear defect reports** (steps, expected result, actual result, screenshots, logs).
- Report defects politely and objectively; focus on the issue, not the person.
- Use a common defect tracking tool so everyone sees the same status.
- Confirm fixes quickly by re-testing.

**How early involvement helps**

- Interface mismatches (data format, field types, units) are found in design itself.
- Developers and testers share the same understanding of requirements.

- Fewer misunderstandings and fewer "not a bug" arguments.
- Integration testing is ready when modules are ready, which saves time.

**Example**

Module A sends date as DD/MM/YYYY while module B expects YYYY-MM-DD. A tester in the interface review notices this and gets it fixed early; otherwise it appears as a failure during integration testing.

**Answer 15**

---

**Concept**

**Defect prevention** means finding the root causes of defects and changing the process so the same defects do not occur again. It is a long-term solution, unlike just fixing each defect.

**Techniques to apply**

1. **Root Cause Analysis (RCA)** on repeated defects, for example using "5 Whys" or a fishbone diagram.
2. **Reviews and inspections** of requirements, design and code.
3. **Coding standards and checklists** prepared from past defects.
4. **Training** of developers and testers on weak areas.
5. **Static analysis tools** and automated checks in the build.
6. **Lessons learned** meeting after each release.
7. Add tests for each repeated defect to the regression suite.

**Example**

If null-value defects come in every release, RCA shows that developers do not check for empty input. A coding checklist item "validate all inputs" is added, reviewers check it, and tests for blank values are added.

**Improvement in quality**

- Fewer defects are injected, so less rework.
- Lower cost than detect-and-fix.
- Stable releases and better customer satisfaction.
- Team learns and the process improves each cycle.

**Answer 16**

**Concept**

The **seven testing principles** guide how testing should be planned. Using them gives a standard strategy for the mobile app.

| **Principle** | **Applied to the mobile app** |
|---|---|
| Testing shows presence of defects | Report defects found and do not claim the app is bug-free |
| Exhaustive testing is impossible | Choose test cases by risk, not every device and input |
| Early testing | Test requirements and UI designs, do unit tests early |
| Defect clustering | Focus on crash-prone modules (camera, payment, sync) |
| Pesticide paradox | Update test cases regularly with new scenarios |
| Testing is context dependent | Test for devices, OS versions, battery, network changes, interruptions (calls) |
| Absence-of-errors fallacy | Check the app meets user needs, not only that it has no defects |

**Standard strategy**

1. Define scope, risks and target devices/OS.
2. Design test cases (functional, usability, compatibility, performance, interruption).
3. Use real devices and emulators.
4. Automate regression tests.
5. Fix defects and re-test; log everything in a tracking tool.
6. Release only when exit criteria are met.

**How it improves reliability**

Same approach is followed every release, risky areas get more attention, and crash causes are tracked, so crashes reduce and the app becomes more reliable.

**Answer 17**

**Concept**

**Testing** finds defects. **Debugging** finds the cause of a defect and fixes it. They are different activities done by different roles.

| **Point** | **Testing** | **Debugging** |
|---|---|---|
| Purpose | Find failures and defects | Locate the cause and fix the defect |
| Done by | Tester (or developer in unit testing) | Developer |
| Starts with | Test case and expected result | A reported defect |
| Output | Defect report, test result | Corrected code |
| Planned? | Yes, planned and can be scheduled | Depends on defects found |
| Can be automated? | Yes | Mostly manual |

**Real-time scenario**

In an online ticket booking app, the tester enters a valid card and the payment fails with "invalid card". This is **testing** (failure found and a defect is logged). The developer then checks the logs, uses a debugger and finds that the card number length check is wrong, and corrects it. This is **debugging**. The tester then **re-tests** the fix.

**Roles in quality**

- Testing shows the quality level and finds problems.
- Debugging removes the problems.
- Both are needed in sequence: Test → Debug → Re-test → Regression test.

**Answer 18**

---

**Concept**

**Cross-browser testing** checks that a web application works the same way on different browsers, versions, operating systems and devices. Different browsers render HTML, CSS and JavaScript differently.

**Strategy (steps)**

1. **Select browsers:** use user analytics to pick popular ones (Chrome, Edge, Firefox, Safari, mobile browsers) and their versions.
2. **Prioritize** based on usage and risk (principle: exhaustive testing is impossible).
3. **Decide what to test:** layout, fonts, images, forms, buttons, JavaScript, payment, cookies and responsive design.
4. **Create test cases** that can be run on every browser.
5. **Set up environment:** real devices, virtual machines or cloud tools like BrowserStack.
6. **Execute and compare** results across browsers.
7. **Log defects** with browser name, version and OS.
8. **Fix and retest;** run regression tests and automate with tools like Selenium.

**Typical issues and handling**

| **Issue** | **Handling** |
|---|---|
| Layout broken in Safari | Fix CSS, use vendor prefixes |
| Button not working in old browser | Use polyfills or supported JavaScript |
| Payment form fails in Firefox | Fix the script, retest on all browsers |
| Mobile screen not fitting | Use responsive design and test on devices |

**Result**

The shopping site gives the same experience to all users, so complaints reduce.

## Answer 19

### Concept

**Input validation** checks that data entered into the system is correct, complete and in the right format. Missing validation lets wrong data enter and spread, which gives inconsistent output.

**Gaps in validation (logistics examples)**

| Gap | Test to find it |
|---|---|
| Negative or zero parcel weight accepted | Boundary and negative tests |
| Wrong pincode length / letters accepted | Equivalence partitioning on format |
| Delivery date earlier than order date | Cross-field validation test |
| Mandatory fields left blank | Blank / null input test |
| Duplicate shipment ID accepted | Duplicate data test |
| Special characters in address crash the system | Error guessing |

**How to find the gaps**

1. Review requirements and business rules for validation rules.
2. List each input field with type, range, length and format.
3. Design tests using Equivalence Partitioning, BVA and negative testing.
4. Run them early (unit and integration level).

**How proper testing improves reliability**

- Wrong data is stopped at the entry point.
- Outputs are consistent and predictable.
- Fewer failures in later modules such as routing and billing.

## Answer 20

### Concept

**Test case design** is the activity of creating test cases from requirements in a planned way. Each test case has an ID, input, steps, expected result and actual result. Testing as an engineering activity means we design before we execute.

**Process to improve**

1. Study requirements and identify test conditions.
2. Choose a design technique (EP, BVA, decision table, state transition).
3. Write test cases with inputs and **expected results**.
4. Review the test cases.
5. Group them into a test suite and map to requirements (RTM).
6. Execute, record results and update the tests.

**Example test case**

| ID | Input | Expected result |
|---|---|---|
| TC01 | Valid user, valid password | Login success |
| TC02 | Valid user, wrong password | Error message shown |
| TC03 | Blank user name | "Required" message |
| TC04 | 3 wrong attempts | Account locked |

**Importance of systematic design**

- Better coverage of requirements.
- Defects found more reliably than by random tests.
- Tests are repeatable and can be reused in regression.
- No duplicate or useless tests, so time is saved.
- Results can be measured and compared.
- New testers can understand and run the tests.

**Answer 21**

---

**Concept**

Data inconsistency between modules is usually an **integration defect**. The same data looks different or is lost when it moves from one module to another.

**Possible root causes**

- Interface mismatch (different data type, format or units).
- Wrong field mapping between modules.

- Data not synchronized or updated in only one place.
- Transaction fails halfway and data is not rolled back.
- Different versions of modules.
- Missing validation at the interface.

**Steps to find the root cause**

1. Reproduce the problem with the same data.
2. Trace the data flow from module to module.
3. Compare data at each interface and in the database.
4. Check logs and the interface specification.
5. Use Root Cause Analysis (for example 5 Whys).
6. Log the defect with the cause.

**Healthcare example**

Patient allergy is saved in the Registration module but is not passed to the Pharmacy module because the field name is different. The doctor sees the allergy but the pharmacy does not. This is dangerous.

**How proper testing ensures consistency**

- Integration testing of every interface.
- Data flow and end-to-end tests (register → treat → prescribe → bill).
- Database consistency checks and rollback tests.
- Regression tests after any module changes.

**Answer 22**

---

**Role of testers across SDLC**

| SDLC phase | Tester's role |
|---|---|
| Requirements | Review for clarity and testability, ask questions, plan acceptance tests |
| Design | Review design and interfaces, prepare test strategy and test plan |
| Coding | Prepare test cases and data, support unit and static testing |
| Testing | Execute tests, report defects, retest, measure coverage |
| Deployment | Support acceptance testing and smoke testing in production-like setup |

| SDLC phase | Tester's role |
|---|---|
| Maintenance | Regression testing and checking each change |

**How early involvement improves quality**

- Defects are found in requirements and design, where they are cheap to fix.
- Test cases are ready by the time coding is complete, so no delay.
- Testers understand the product better.
- Developers and testers share the same understanding; less rework.
- The project gets finished on time with better quality.

**Link to the scenario**

The project was delayed because all testing started late and many defects came together. If testers had joined from the start, defects would have come in small numbers throughout, and the delay would not happen.

**Answer 23**

---

**Concept**

**Test coverage** shows how much of the software (requirements, scenarios or code) is covered by tests. Formula: Coverage = (items tested / total items) x 100.

**Gaps in the scenario**

- No load, stress or scalability tests for peak usage.
- No concurrency tests (many users doing the same action).
- Only normal flows tested; rare and critical cases missed.
- No risk analysis to find high-risk scenarios.

**Principles applied**

| Principle | Use here |
|---|---|
| Exhaustive testing is impossible | Pick the most important peak-load scenarios using risk |
| Testing is context dependent | Use performance testing for this kind of system |
| Early testing | Define the expected load in the requirements |

| Principle | Use here |
|---|---|
| Defect clustering | Test heavily the modules which fail under load |

**How better coverage prevents failure**

1. Prepare a list of scenarios including peak, stress and failure conditions.
2. Map each to test cases (RTM) to find uncovered requirements.
3. Run load and stress tests with expected and higher users.
4. Fix bottlenecks and retest.
5. Track coverage until the exit criteria are met.

**Example**

A sale starts and 10,000 users log in at the same time. A load test with 10,000 and 15,000 users before the sale would show slow response or crash earlier.

**Answer 24**

---

**Concept**

A **defect repository** stores all defect reports in one place. Proper documentation gives each defect a record that can be tracked until closure.

**Details to record**

| Field | Purpose |
|---|---|
| Defect ID, title | Unique identification |
| Module / version | Where it was found |
| Steps to reproduce | Helps developer reproduce |
| Expected vs actual result | Shows the problem |
| Severity, priority | Decides urgency |
| Status, assigned to | Tracking and ownership |
| Phase found, root cause | Analysis and prevention |
| Attachments (screenshot, logs) | Evidence |

**Defect life cycle for tracking**

New → Assigned → Open → Fixed → Retest → Closed (or Reopened / Rejected / Deferred).

**How documentation helps**

- **Tracking:** everyone knows the status and owner of each defect.
- **Analysis:** defect density, defects per module, trend and root cause reports.
- **Communication:** less confusion between tester and developer.
- **Reuse:** past defects help in test design and regression.
- **Management:** helps to decide release readiness.

**Solution for the team**

Use a tool such as JIRA or Bugzilla, define a standard defect template and make it mandatory to log every defect.

**Answer 25**

---

**Concept**

Repeated failures show that testing was not planned using the basic testing principles. The approach is redesigned using the principles.

| **Principle** | **Banking testing approach** |
|---|---|
| Testing shows presence of defects | Report all failures; do not claim zero defects |
| Exhaustive testing impossible | Risk-based testing: funds transfer, interest, EMI, ATM first |
| Early testing | Review requirements and business rules before coding |
| Defect clustering | More tests for modules with past failures |
| Pesticide paradox | Update test cases every release |
| Context dependent | Security, accuracy and concurrency tests for banking |
| Absence-of-errors fallacy | Validate with business users (UAT) |

**Structured approach**

1. Test plan with scope, risk and exit criteria.

2. Test design using EP, BVA, decision tables and state transition.
3. Test types: functional, integration (payment gateway), security, performance, rollback and regression.
4. Defect tracking and reporting.
5. Regression suite after each fix.

**How it improves reliability**

Transactions are tested for normal, boundary, failure and concurrent cases, so repeated failures are found before release. Past failures are added to the regression suite so they do not return.

**Answer 26**

---

**Concept**

**Edge cases** are extreme or unusual inputs such as minimum, maximum, empty, zero or very long values. Testing only normal inputs leaves these untested.

**Principles applied**

- **Exhaustive testing is impossible:** so choose the inputs most likely to fail, which are the edges.
- **Defect clustering:** many defects occur at limits.
- Use **Equivalence Partitioning and Boundary Value Analysis** to select inputs.

**Edge cases to include**

| **Input type** | **Edge cases** |
|---|---|
| Number field (1 to 100) | 0, 1, 2, 99, 100, 101, -1 |
| Text field (max 20) | Empty, 1 char, 20, 21 chars, spaces, special characters |
| Date | 29 February, 31 December, past and future dates |
| Amount | 0.00, 0.01, maximum limit, limit + 1 |

**How boundary testing improves detection**

- Developers often make mistakes with <, <=, >, >=, so errors appear exactly at the boundaries.
- A few chosen tests (6 per range) give high chance of finding defects.

- Saves time because no need to test all values.
- Finds crashes, wrong messages and wrong calculations early.

**Example**

Transfer amount allowed Rs. 1 to Rs. 50,000. Tests: 0, 1, 2, 49,999, 50,000, 50,001. A defect where 50,000 is rejected will be found by the test of 50,000.

**Answer 27**

---

**Concept**

**Process-based testing** means all teams follow the same defined test process, templates and rules. This is called standardization.

**Approaches to apply**

1. Define an organization-wide **test process**: planning, analysis and design, implementation and execution, exit criteria evaluation, closure.
2. Use **standard templates** for test plan, test cases and defect reports.
3. Use the **same severity and priority scale** for all modules.
4. Define **entry and exit criteria** for every test level.
5. Hold **reviews** of test cases and test results.
6. Collect the same **metrics** (coverage, defect density) for every module.
7. Train the team on the process and follow a standard such as ISO/IEC 29119 or a maturity model like TMMi.

**How standardization improves outcomes**

| **Before** | **After** |
|---|---|
| Each module tested differently | All modules follow the same steps |
| Uneven quality | Consistent quality |
| Hard to compare results | Same metrics, easy comparison |
| New members confused | Easy to learn and follow |
| Missed or repeated work | Clear tasks and responsibilities |

**Answer 28**

---

**Concept**

**Input validation** checks that data is of the correct type, length, range and format before the system uses it. **Robustness** is the ability of software to handle invalid or unexpected input without crashing.

**Missed validation scenarios and tests**

| **Input** | **Expected behaviour** |
|---|---|
| Empty / blank field | Shows a "required" message |
| Letters in a numeric field | Rejected with message |
| Very long text | Limited or rejected, no crash |
| Special characters, e.g. <>'" | Handled safely, no error page |
| SQL or script text (injection) | Blocked, no data exposed |
| Negative or zero values | Rejected as per rules |
| Spaces only | Treated as empty |

**How to improve**

1. List all input fields with the rules for each.
2. Design **negative test cases** using EP, BVA and error guessing.
3. Check validation on both **client side and server side**.
4. Check that error messages are clear and the application keeps running.
5. Retest and add the cases to regression.

**How proper testing ensures robustness**

- The system is checked against wrong inputs before real users enter them.
- Crashes and security problems are found in testing.
- Users get helpful messages, and data remains correct.

**Answer 29**

---

**Concept**

**Defect leakage** means defects that escaped testing and were found in production (by customers).

Defect leakage % = (defects found in production / (defects found in testing + production)) x 100

Example: 40 defects found in testing and 10 in production gives 10/50 x 100 = 20%.

**Root cause analysis**

| **Possible cause** | **Explanation** |
|---|---|
| Incomplete requirements | Tests were not designed for missing scenarios |
| Poor test coverage | Some features or paths never tested |
| Weak test cases | Old tests used again (pesticide paradox) |
| No or little regression testing | Changes broke old features |
| Test environment different from production | Data or configuration issues not seen |
| Time pressure | Testing was cut short |
| No UAT | Business use cases not checked |

**Steps to analyze**

1. Collect production defects.
2. Find which phase should have caught each.
3. Group the causes.
4. Take action for each cause.

**How improved testing prevents leakage**

- Review requirements and improve test design.
- Increase coverage using RTM and risk analysis.
- Run regression tests after every change.
- Make the test environment similar to production.

- Do UAT before release.
- Add every leaked defect as a new test case.

**Answer 30**

---

**Concept**

The problem is late defect detection. The strategy is redesigned using the **early testing principle (shift-left)** together with risk-based testing and defect clustering.

**Redesigned strategy**

1. **Review requirements and design** (static testing) with testers involved.
2. **Prepare test plan and test cases early,** parallel to development.
3. **Unit testing and code reviews** by developers.
4. **Continuous integration** with automated smoke tests on every build.
5. **Risk-based testing:** high-risk modules first.
6. **Track defects and the phase found** to improve the process.

**Relative cost of defects**

| **Phase found** | **Relative cost to fix** |
|---|---|
| Requirements | 1x |
| Design | 5x |
| Coding / unit test | 10x |
| System test | 20x to 50x |
| Production | 100x or more |

(These values are a general guide to show how cost increases, not exact figures.)

**How early detection improves efficiency**

- Lower cost and less rework.
- Defects do not spread to later phases.
- Shorter testing cycle and fewer delays.
- Team can focus later testing on new risks.
- Better quality at release and happier customers.

# Code of Conduct:

I ____**G Bhumika/192312237**____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

**Signature of the student:**

G Bhumika